10 ejercicios de Python: Funciones + Estructuras de datos

Oct 18, 2026 · @Cesar Moreno

Cómo usar esta guía

Esta guía reúne 10 ejercicios que combinan funciones con listas, tuplas y diccionarios, todos sobre el inventario de TecnoMarket de la evaluación; cada uno trae su solución y una explicación línea por línea.

Método sugerido para el aprendiz:

1. Lee el enunciado e intenta resolverlo solo, sin mirar la solución.
2. Compara tu código con la solución y lee la tabla de explicación.
3. Ejecuta el código, cambia los datos (stock, precios, un producto nuevo) y predice la salida antes de correrlo.
4. Resuelve el apartado «Para pensar» de cada ejercicio.

Datos base

Copia este bloque al inicio de cada práctica; los ejercicios 4 al 10 usan el diccionario `inventario`, que se construye con la función de la evaluación.

```
productos = [
    ("Mouse", 25000, 10),
    ("Teclado", 60000, 3),
    ("Monitor", 450000, 5),
    ("Audifonos", 80000, 2),
    ("Cable HDMI", 15000, 20),
]
# Cada tupla: (nombre, precio, stock)

ventas = [("Mouse", 3), ("Monitor", 6), ("Teclado", 1), ("Parlante", 2),
("Mouse", 4)]
# Cada tupla: (producto, cantidad)

def crear_inventario(productos):
    inventario = {}
    for nombre, precio, stock in productos:
```

```
        inventario[nombre] = {"precio": precio, "stock": stock}
    return inventario

inventario = crear_inventario(productos)
# {"Mouse": {"precio": 25000, "stock": 10}, "Teclado": {...}, ...}
```

Recordatorio de las estructuras

Estructura	Se escribe con	¿Se puede modificar?	Ejemplo en TecnoMarket
Lista	`[ ]`	Sí	`productos`, la lista completa de registros
Tupla	`( )`	No	`("Mouse", 25000, 10)`, un registro fijo
Diccionario	`{ clave: valor }`	Sí	`{"precio": 25000, "stock": 10}`
Diccionario anidado	un diccionario dentro de otro	Sí	`inventario["Mouse"]["stock"]`

Reglas que se repiten en todos los ejercicios

- Toda función recibe sus datos por parámetro; no usa variables globales dentro.
- `return` entrega un valor que se puede guardar; `print` solo lo muestra en pantalla.
- Una función sin `return` devuelve `None`.

Mapa de los 10 ejercicios

N.º	Función	Estructuras que practica	Nivel
1	`obtener_nombres`	Lista de tuplas → lista	Básico
2	`crear_catalogo`	Lista de tuplas → diccionario	Básico
3	`estadisticas_precios`	Devolver una tupla	Básico
4	`consultar_stock`	`.get()` y parámetro por defecto	Básico

N.º	Función	Estructuras que practica	Nivel
5	`filtrar_por_precio`	Parámetros por defecto, lista de tuplas	Intermedio
6	`unidades_vendidas`	Diccionario acumulador	Intermedio
7	`reponer_stock`	Copiar diccionarios anidados	Intermedio
8	`top_valor`	`sorted`, `lambda` y rebanado	Intermedio
9	`clasificar_por_stock`	Diccionario de listas	Intermedio
10	`cotizar_pedido` e `imprimir_cotizacion`	Todo lo anterior	Integrador

Ejercicio 1 · Extraer los nombres de los productos

Nivel: básico. Esta función recorre una lista de tuplas y devuelve una lista nueva con solo el primer dato de cada tupla.

Enunciado. Escribe la función `obtener_nombres(productos)` que reciba la lista de tuplas `(nombre, precio, stock)` y devuelva una lista con los nombres, en el mismo orden.

Conceptos que practicas: recorrer una lista con `for`, desempaquetar una tupla, agregar a una lista con `.append()` y devolver con `return`.

Solución

```
def obtener_nombres(productos):
    nombres = []
    for nombre, precio, stock in productos:
        nombres.append(nombre)
    return nombres
```

Prueba

```
print(obtener_nombres(productos))
```

Salida esperada

```
['Mouse', 'Teclado', 'Monitor', 'Audifonos', 'Cable HDMI']
```

Explicación línea por línea

Línea	Código	Qué hace y por qué
1	`def obtener_nombres(productos):`	Define la función. `def` es la palabra reservada para crear funciones; `productos` es el parámetro: el nombre con el que la función llamará a la lista que reciba. Los dos puntos al final abren el bloque de código de la función.
2	`nombres = []`	Crea una lista vacía donde se irán guardando los resultados. Se crea antes del ciclo; si estuviera dentro, se vaciaría en cada vuelta.
3	`for nombre, precio, stock in productos:`	Recorre la lista una tupla por vez. En cada vuelta Python desempaqueta la tupla: el primer valor va a `nombre`, el segundo a `precio` y el tercero a `stock`. Por eso el patrón de la izquierda debe tener exactamente tres variables, igual que las tuplas.
4	`nombres.append(nombre)`	Agrega `nombre` al final de la lista `nombres`. Está con sangría de 8 espacios porque pertenece al `for`: se ejecuta una vez por cada tupla. `precio` y `stock` no se usan aquí, y está bien.
5	`return nombres`	Devuelve la lista completa. Tiene sangría de 4 espacios (dentro de la función, pero fuera del `for`), así que se ejecuta una sola vez, cuando el ciclo ya terminó.

Seguimiento paso a paso

Vuelta	Tupla recibida	`nombre`	`nombres` después de `.append`
1	`("Mouse", 25000, 10)`	`"Mouse"`	`['Mouse']`
2	`("Teclado", 60000, 3)`	`"Teclado"`	`['Mouse', 'Teclado']`

Vuelta	Tupla recibida	`nombre`	`nombres` después de `.append`
3	`("Monitor", 450000, 5)`	`"Monitor"`	`['Mouse', 'Teclado', 'Monitor']`
4	`("Audifonos", 80000, 2)`	`"Audifonos"`	`['Mouse', 'Teclado', 'Monitor', 'Audifonos']`
5	`("Cable HDMI", 15000, 20)`	`"Cable HDMI"`	`['Mouse', 'Teclado', 'Monitor', 'Audifonos', 'Cable HDMI']`

Error frecuente. Escribir `return nombres` con la misma sangría que `nombres.append(nombre)`. La función devolvería la lista después de la primera vuelta y solo traería `['Mouse']`.

Para pensar. ¿Cómo cambiarías la función para devolver solo los nombres de los productos con stock mayor que 5?

Ejercicio 2 · Construir un catálogo nombre → precio

Nivel: básico. Esta función convierte una lista de tuplas en un diccionario, para encontrar el precio de un producto por su nombre sin recorrer toda la lista.

Enunciado. Escribe la función `crear_catalogo(productos)` que reciba la lista de tuplas `(nombre, precio, stock)` y devuelva un diccionario donde la clave sea el nombre y el valor sea el precio.

Conceptos que practicas: crear un diccionario vacío, asignar pares clave-valor, desempaquetar tuplas y consultar un diccionario.

Solución

```
def crear_catalogo(productos):
    catalogo = {}
    for nombre, precio, stock in productos:
        catalogo[nombre] = precio
    return catalogo
```

Prueba

```
catalogo = crear_catalogo(productos)
print(catalogo)
print(catalogo["Monitor"])
print(catalogo.get("Parlante", "No disponible"))
```

Salida esperada

```
{'Mouse': 25000, 'Teclado': 60000, 'Monitor': 450000, 'Audifonos': 80000,
'Cable HDMI': 15000}
450000
No disponible
```

Explicación línea por línea

Línea	Código	Qué hace y por qué
1	`def crear_catalogo(productos):`	Define la función con un parámetro: la lista de tuplas. Los datos llegan por parámetro, no se leen de una variable global.
2	`catalogo = {}`	Crea un diccionario vacío. Las llaves `{}` sin contenido son un diccionario, no un conjunto.
3	`for nombre, precio, stock in productos:`	Recorre las tuplas y las desempaqueta en tres variables, igual que en el ejercicio 1.
4	`catalogo[nombre] = precio`	Guarda un par en el diccionario: la clave es el contenido de `nombre` y el valor es el de `precio`. Si la clave no existe, se crea; si ya existía, se reemplaza su valor. El `stock` se ignora porque el catálogo solo guarda precios.
5	`return catalogo`	Entrega el diccionario completo cuando el ciclo terminó.

Explicación de la prueba

Código	Qué hace
`catalogo = crear_catalogo(productos)`	Llama a la función y guarda en `catalogo` el diccionario que ella devuelve.
`catalogo["Monitor"]`	Busca por clave y devuelve `450000`. Si la clave no existiera, lanzaría `KeyError`.
`catalogo.get("Parlante", "No disponible")`	Busca `"Parlante"`; como no existe, no falla y devuelve el segundo argumento, el valor por defecto.

Error frecuente. Escribir `catalogo.append(nombre, precio)`. Los diccionarios no tienen `.append()`; se agrega un par con `catalogo[clave] = valor`.

Para pensar. Si la lista tuviera dos tuplas con el nombre `"Mouse"` y precios distintos, ¿cuál precio quedaría en el catálogo y por qué?

Ejercicio 3 · Estadísticas de precios con una tupla de retorno

Nivel: básico. Una función solo puede devolver un objeto, pero ese objeto puede ser una tupla con varios valores; así una función entrega varios resultados a la vez.

Enunciado. Escribe la función `estadisticas_precios(productos)` que reciba la lista de tuplas y devuelva una tupla `(minimo, maximo, promedio)` con el precio menor, el precio mayor y el precio promedio.

Conceptos que practicas: construir una lista a partir de otra, usar `min`, `max`, `sum` y `len`, devolver varios valores y desempaquetar el resultado.

Solución

```
def estadisticas_precios(productos):
    precios = []
    for nombre, precio, stock in productos:
        precios.append(precio)
    minimo = min(precios)
    maximo = max(precios)
    promedio = sum(precios) / len(precios)
    return minimo, maximo, promedio
```

Prueba

```
resultado = estadisticas_precios(productos)
print(resultado)
print(type(resultado))

menor, mayor, prom = estadisticas_precios(productos)
print(f"Menor: ${menor:,} - Mayor: ${mayor:,} - Promedio: ${prom:,.0f}")
```

Salida esperada

```
(15000, 450000, 126000.0)
<class 'tuple'>
Menor: $15,000 - Mayor: $450,000 - Promedio: $126,000
```

Explicación línea por línea

Línea	Código	Qué hace y por qué
1	`def estadisticas_precios(productos):`	Define la función; recibe la lista de tuplas por parámetro.
2	`precios = []`	Lista vacía donde se guardarán solo los precios. Separarlos permite aplicar `min`, `max` y `sum`, que trabajan sobre listas de números.
3	`for nombre, precio, stock in productos:`	Recorre cada tupla y la desempaqueta en tres variables.
4	`precios.append(precio)`	Agrega el precio de cada producto a la lista. Al terminar el ciclo, `precios` vale `[25000, 60000, 450000, 80000, 15000]`.
5	`minimo = min(precios)`	`min` devuelve el menor elemento de la lista: `15000`.
6	`maximo = max(precios)`	`max` devuelve el mayor elemento: `450000`.

Línea	Código	Qué hace y por qué
7	`promedio = sum(precios) / len(precios)`	`sum` suma todos los precios (`630000`) y `len` cuenta cuántos hay (`5`). La división `/` siempre da un número decimal, por eso el resultado es `126000.0` .
8	`return minimo, maximo, promedio`	Al separar valores con comas, Python los empaqueta automáticamente en una tupla `(15000, 450000, 126000.0)` . No hacen falta paréntesis.

Explicación de la prueba

Código	Qué hace
`resultado = estadisticas_precios(productos)`	Guarda la tupla completa en una sola variable.
`type(resultado)`	Confirma que lo devuelto es una `tuple` ; es la misma idea de la pregunta 3 de la Parte II de la evaluación.
`menor, mayor, prom = estadisticas_precios(productos)`	Desempaqueta la tupla: el primer valor va a `menor` , el segundo a `mayor` y el tercero a `prom` . Debe haber tantas variables como valores devueltos.
`{menor:,}`	Dentro del f-string, `:,` muestra el número con separador de miles.
`{prom:,.0f}`	Muestra el número con separador de miles y sin decimales (`.0f`).

Error frecuente. Poner `print(minimo, maximo, promedio)` en lugar de `return` . La función mostraría los datos, pero quien la llame recibiría `None` y no podría usar los valores.

Para pensar. Si desempaquetas con `a, b = estadisticas_precios(productos)` , ¿qué error aparece y por qué?

Ejercicio 4 · Consultar el stock sin que el programa falle

Nivel: básico. Buscar una clave que no existe provoca un `KeyError`; esta función lo evita con `.get()` y ofrece un valor por defecto que el usuario puede cambiar.

Enunciado. Escribe la función `consultar_stock(inventario, nombre, defecto=0)` que devuelva el stock del producto indicado. Si el producto no existe en el inventario, debe devolver el valor de `defecto`.

Conceptos que practicas: diccionario anidado, método `.get()`, comparación con `None`, parámetro con valor por defecto y llamada con argumento por nombre.

Solución

```
def consultar_stock(inventario, nombre, defecto=0):
    datos = inventario.get(nombre)
    if datos is None:
        return defecto
    return datos["stock"]
```

Prueba

```
print(consultar_stock(inventario, "Mouse"))
print(consultar_stock(inventario, "Parlante"))
print(consultar_stock(inventario, "Parlante", -1))
print(consultar_stock(inventario, "Teclado", defecto=99))
```

Salida esperada

```
10
0
-1
3
```

Explicación línea por línea

Línea	Código	Qué hace y por qué
1	`def consultar_stock(inventario, nombre, defecto=0):`	Define la función con tres parámetros. `inventario` y `nombre` son obligatorios. `defecto=0` es opcional: si quien llama no lo envía, vale `0`. Los parámetros con valor por defecto siempre van al final de la lista.
2	`datos = inventario.get(nombre)`	Busca `nombre` en el diccionario. Si existe, `datos` recibe el diccionario interno, por ejemplo `{"precio": 25000, "stock": 10}`. Si no existe, `.get()` no falla y devuelve `None`.
3	`if datos is None:`	Pregunta si la búsqueda no encontró nada. Con `None` se usa `is`, no `==`, porque `None` es un único objeto especial.
4	`return defecto`	Si el producto no existe, termina la función y devuelve el valor por defecto. Al ejecutarse un `return`, las líneas siguientes ya no se leen.
5	`return datos["stock"]`	Solo se llega aquí si el producto existe. Entra al diccionario interno y devuelve el valor de la clave `"stock"`.

Explicación de las llamadas

Llamada	Resultado	Por qué
`consultar_stock(inventario, "Mouse")`	`10`	El producto existe; se devuelve su stock.
`consultar_stock(inventario, "Parlante")`	`0`	No existe y no se envió `defecto`, así que se usa `0`.

Llamada	Resultado	Por qué
`consultar_stock(inventario, "Parlante", -1)`	`-1`	No existe; el tercer argumento reemplaza el valor por defecto.
`consultar_stock(inventario, "Teclado", defecto=99)`	`3`	Se envía `defecto` por nombre. Como el producto existe, ese valor no se usa.

Error frecuente. Escribir `inventario[nombre]["stock"]` directamente. Con un producto inexistente lanza `KeyError` y el programa se detiene.

Para pensar. ¿Por qué `.get(nombre)` no devuelve directamente el stock? Piensa en qué contiene el valor guardado bajo cada nombre.

Ejercicio 5 · Filtrar productos por rango de precio

Nivel: intermedio. La función combina un diccionario anidado, condiciones y dos parámetros opcionales, y devuelve una lista de tuplas con los productos que cumplen.

Enunciado. Escribe la función `filtrar_por_precio(inventario, minimo=0, maximo=None)` que devuelva una lista de tuplas `(nombre, precio)` de los productos cuyo precio sea mayor o igual que `minimo` y menor o igual que `maximo`. Si `maximo` no se envía (`None`), no hay tope superior.

Conceptos que practicas: recorrer un diccionario con `.items()`, condiciones con `and` y `or`, parámetros por defecto y creación de tuplas.

Solución

```
def filtrar_por_precio(inventario, minimo=0, maximo=None):
    resultado = []
    for nombre, datos in inventario.items():
        precio = datos["precio"]
        cumple_minimo = precio >= minimo
        cumple_maximo = maximo is None or precio <= maximo
        if cumple_minimo and cumple_maximo:
            resultado.append((nombre, precio))
    return resultado
```

Prueba

```
print(filtrar_por_precio(inventario, 20000, 100000))
print(filtrar_por_precio(inventario, minimo=100000))
print(len(filtrar_por_precio(inventario)))
```

Salida esperada

```
[('Mouse', 25000), ('Teclado', 60000), ('Audifonos', 80000)]
[('Monitor', 450000)]
5
```

Explicación línea por línea

Línea	Código	Qué hace y por qué
1	`def filtrar_por_precio(inventario, minimo=0, maximo=None):`	Define la función. `minimo=0` significa que, si no se indica, se aceptan todos los precios desde cero. `maximo=None` usa `None` como señal de «sin tope», porque cualquier número que se eligiera como tope podría quedarse corto.
2	`resultado = []`	Lista vacía para guardar las tuplas que cumplan el filtro.
3	`for nombre, datos in inventario.items():`	`.items()` entrega cada par del diccionario como una tupla `(clave, valor)`. Se desempaquetan en `nombre` (el texto) y `datos` (el diccionario interno con precio y stock).
4	`precio = datos["precio"]`	Saca el precio del diccionario interno y lo guarda en una variable corta, para no repetir `datos["precio"]` más abajo.
5	`cumple_minimo = precio >= minimo`	La comparación produce `True` o `False`, y se guarda con un nombre que explica su intención.

Línea	Código	Qué hace y por qué
6	`cumple_maximo = maximo is None or precio <= maximo`	Es `True` si no hay tope (`maximo is None`) o si el precio no lo supera. Si `maximo` es `None`, Python ya no evalúa lo que está después del `or`; eso evita comparar un número con `None`, que causaría un `TypeError`.
7	`if cumple_minimo and cumple_maximo:`	Solo entra al bloque si las dos condiciones son verdaderas al mismo tiempo.
8	`resultado.append((nombre, precio))`	Agrega una tupla a la lista. Hay dos pares de paréntesis: los externos son de `.append()` y los internos crean la tupla. Sin los internos, `.append` recibiría dos argumentos y fallaría.
9	`return resultado`	Devuelve la lista, con la sangría del `def` para que se ejecute una sola vez, después del ciclo.

Seguimiento de la primera prueba (`minimo=20000`, `maximo=100000`)

Producto	Precio	¿≥ 20000?	¿≤ 100000?	¿Se agrega?
Mouse	25000	Sí	Sí	Sí
Teclado	60000	Sí	Sí	Sí
Monitor	450000	Sí	No	No
Audifonos	80000	Sí	Sí	Sí
Cable HDMI	15000	No	Sí	No

Error frecuente. Poner `maximo=100000` como valor por defecto. Funcionaría con estos datos, pero la función dejaría de servir si algún producto costara más; `None` expresa mejor «sin límite».

Para pensar. ¿Qué cambiaría si quisieras excluir los productos que cuestan exactamente `minimo`? ¿En qué línea harías el cambio?

Ejercicio 6 · Contar unidades vendidas por producto

Nivel: intermedio. Cuando un mismo producto aparece varias veces en una lista, un diccionario acumulador permite sumar sus cantidades en un solo lugar.

Enunciado. Escribe la función `unidades_vendidas(ventas)` que reciba la lista de ventas `(producto, cantidad)` y devuelva un diccionario con el total de unidades vendidas de cada producto.

Conceptos que practicas: diccionario como acumulador, `.get(clave, 0)` para iniciar un contador, desempaquetado de tuplas y lista vacía como caso límite.

Solución

```
def unidades_vendidas(ventas):
    acumulado = {}
    for producto, cantidad in ventas:
        acumulado[producto] = acumulado.get(producto, 0) + cantidad
    return acumulado
```

Prueba

```
totales = unidades_vendidas(ventas)
print(totales)
print(totales["Mouse"])
print(unidades_vendidas([]))
```

Salida esperada

```
{'Mouse': 7, 'Monitor': 6, 'Teclado': 1, 'Parlante': 2}
7
{}
```

Explicación línea por línea

Línea	Código	Qué hace y por qué
1	`def unidades_vendidas(ventas):`	Define la función; recibe la lista de ventas por parámetro.
2	`acumulado = {}`	Diccionario vacío que irá guardando el total de cada producto. Empieza sin ninguna clave.

Línea	Código	Qué hace y por qué
3	`for producto, cantidad in ventas:`	Recorre las ventas y desempaqueta cada tupla de dos valores en `producto` y `cantidad`.
4	`acumulado[producto] = acumulado.get(producto, 0) + cantidad`	Es la línea clave y se lee de derecha a izquierda. Primero `acumulado.get(producto, 0)` busca lo que ya se había acumulado para ese producto, y devuelve `0` si es la primera vez que aparece. Luego se le suma `cantidad`. Finalmente el resultado se guarda de nuevo bajo la misma clave.
5	`return acumulado`	Devuelve el diccionario con los totales una vez procesadas todas las ventas.

Seguimiento paso a paso

Vuelta	Venta	Valor previo (`.get`)	Nuevo valor	`acumulado` queda
1	`("Mouse", 3)`	0 (no existía)	3	`{'Mouse': 3}`
2	`("Monitor", 6)`	0 (no existía)	6	`{'Mouse': 3, 'Monitor': 6}`
3	`("Teclado", 1)`	0 (no existía)	1	`{'Mouse': 3, 'Monitor': 6, 'Teclado': 1}`
4	`("Parlante", 2)`	0 (no existía)	2	`{'Mouse': 3, 'Monitor': 6, 'Teclado': 1, 'Parlante': 2}`
5	`("Mouse", 4)`	3	7	`{'Mouse': 7, 'Monitor': 6, 'Teclado': 1, 'Parlante': 2}`

En la vuelta 5 el Mouse ya existía, así que su total pasa de 3 a 7 sin cambiar de posición en el diccionario.

Error frecuente. Escribir `acumulado[producto] += cantidad` sin haber creado antes la clave. La primera vez que aparece un producto lanza `KeyError`, porque no hay un valor al que sumarle.

Para pensar. Esta función cuenta también `"Parlante"`, que no está en el inventario. ¿Qué parámetro agregarías y qué condición escribirías para ignorar los productos que no existen?

Ejercicio 7 · Reponer stock sin modificar el inventario original

Nivel: intermedio. Los diccionarios son mutables: si una función los cambia, el cambio se ve fuera de ella. Esta función devuelve un inventario nuevo y deja el original intacto.

Enunciado. Escribe la función `reponer_stock(inventario, reposicion)` donde `reposicion` es una lista de tuplas `(producto, cantidad)`. Debe devolver un inventario nuevo con el stock aumentado en esas cantidades. Los productos que no existan en el inventario se ignoran. El inventario original no debe cambiar.

Conceptos que practicas: diccionarios anidados, copia de estructuras mutables, operador `in` con diccionarios y `+=` sobre un valor interno.

Solución

```
def reponer_stock(inventario, reposicion):
    nuevo = {}
    for nombre, datos in inventario.items():
        nuevo[nombre] = {"precio": datos["precio"], "stock": datos["stock"]}
    for nombre, cantidad in reposicion:
        if nombre in nuevo:
            nuevo[nombre]["stock"] += cantidad
    return nuevo
```

Prueba

```
reposicion = [("Teclado", 10), ("Audifonos", 5), ("Parlante", 3)]
nuevo = reponer_stock(inventario, reposicion)
print(nuevo["Teclado"]["stock"])
print(nuevo["Audifonos"]["stock"])
print("Parlante" in nuevo)
print(inventario["Teclado"]["stock"])
```

Salida esperada

```
13
7
False
3
```

Explicación línea por línea

Línea	Código	Qué hace y por qué
1	`def reponer_stock(inventario, reposicion):`	Define la función con dos parámetros: el inventario y la lista de reposiciones.
2	`nuevo = {}`	Diccionario vacío que será la copia. Trabajar sobre una copia protege los datos originales.
3	`for nombre, datos in inventario.items():`	Recorre cada producto del inventario original. `datos` es el diccionario interno de ese producto.
4	`nuevo[nombre] = {"precio": datos["precio"], "stock": datos["stock"]}`	Crea un diccionario interno nuevo con los mismos valores. Es lo que hace independiente la copia: `nuevo` y `inventario` ya no comparten los diccionarios internos. Si se asignara `datos` directamente, ambos apuntarían al mismo objeto.
5	`for nombre, cantidad in reposicion:`	Segundo ciclo, ya terminado el primero (misma sangría que la línea 3). Recorre las reposiciones y desempaqueta cada tupla.
6	`if nombre in nuevo:`	Con diccionarios, `in` revisa las claves. Si el producto no existe, la reposición se ignora y se evita un `KeyError`.
7	`nuevo[nombre]["stock"] += cantidad`	Entra en dos niveles: `nuevo[nombre]` es el diccionario interno y `["stock"]` es su stock. `+=` suma `cantidad` a lo que ya había. Solo se modifica la copia.

Línea	Código	Qué hace y por qué
8	`return nuevo`	Devuelve el inventario actualizado.

Qué pasa con cada reposición

Reposición	¿Existe?	Stock antes	Stock en `nuevo`	Stock en `inventario`
`("Teclado", 10)`	Sí	3	13	3
`("Audifonos", 5)`	Sí	2	7	2
`("Parlante", 3)`	No	no aplica	se ignora	no aplica

Error frecuente. Escribir `nuevo = inventario` o `nuevo = inventario.copy()`. La primera no crea ninguna copia, solo otro nombre para el mismo diccionario. La segunda copia el diccionario externo, pero los diccionarios internos siguen compartidos, así que `nuevo["Teclado"]["stock"] += 10` también cambiaría el original.

Para pensar. ¿Qué devolvería la función si `reposicion` fuera una lista vacía? ¿Sería el mismo diccionario o uno igual pero independiente?

Ejercicio 8 · Los N productos que más valor tienen en bodega

Nivel: intermedio. Para ordenar una lista de tuplas por un dato que no es el primero, se le indica a `sorted` cuál usar con el parámetro `key`.

Enunciado. Escribe la función `top_valor(inventario, n=3)` que calcule el valor en bodega de cada producto (`precio × stock`) y devuelva una lista de tuplas `(nombre, valor)` con los `n` productos de mayor valor, del más alto al más bajo.

Conceptos que practicas: recorrer un diccionario anidado, calcular valores, `sorted` con `key` y `reverse`, función `lambda` y rebanado de listas.

Solución

```
def top_valor(inventario, n=3):
    valores = []
    for nombre, datos in inventario.items():
        valor = datos["precio"] * datos["stock"]
        valores.append((nombre, valor))
    ordenados = sorted(valores, key=lambda par: par[1], reverse=True)
    return ordenados[:n]
```

Prueba

```
print(top_valor(inventario))
print(top_valor(inventario, 1))
print(len(top_valor(inventario, 10)))
```

Salida esperada

```
[('Monitor', 2250000), ('Cable HDMI', 300000), ('Mouse', 250000)]
[('Monitor', 2250000)]
5
```

Explicación línea por línea

Línea	Código	Qué hace y por qué
1	`def top_valor(inventario, n=3):`	Define la función. `n=3` hace que, por defecto, se devuelvan los tres mejores.
2	`valores = []`	Lista vacía que guardará tuplas `(nombre, valor)`.
3	`for nombre, datos in inventario.items():`	Recorre cada producto: `nombre` es la clave y `datos` es el diccionario interno.
4	`valor = datos["precio"] * datos["stock"]`	Calcula cuánto dinero hay en bodega de ese producto. Para el Monitor: 450000 × 5 = 2250000.

Línea	Código	Qué hace y por qué
5	`valores.append((nombre, valor))`	Agrega una tupla de dos elementos a la lista. Al terminar el ciclo, `valores` vale `[('Mouse', 250000), ('Teclado', 180000), ('Monitor', 2250000), ('Audifonos', 160000), ('Cable HDMI', 300000)]`.
6	`ordenados = sorted(valores, key=lambda par: par[1], reverse=True)`	`sorted` crea una lista nueva ordenada y no modifica `valores`. `key` indica por qué dato ordenar. `lambda par: par[1]` es una función pequeña sin nombre: recibe cada tupla (`par`) y devuelve su posición 1, el valor. Sin `key` , ordenaría por nombre. `reverse=True` ordena de mayor a menor.
7	`return ordenados[:n]`	El rebanado `[:n]` toma los primeros `n` elementos. Si `n` es mayor que la cantidad de productos, no falla: devuelve todos.

La misma línea 6, sin `lambda`

Una `lambda` es solo una forma corta de escribir una función. Estas dos versiones hacen lo mismo:

```
def segundo_elemento(par):
    return par[1]

ordenados = sorted(valores, key=segundo_elemento, reverse=True)
```

Control de resultados. La suma de los cinco valores es 3140000, el mismo resultado que da `valor_total` en la evaluación; sirve para comprobar que el cálculo de la línea 4 es correcto.

Error frecuente. Escribir `valores.sort(...)` y guardar su resultado en una variable. `.sort()` modifica la lista en su lugar y devuelve `None` ; por eso aquí se usa `sorted` , que sí devuelve la lista ordenada.

Para pensar. Si dos productos tuvieran el mismo valor, ¿cuál aparecería primero? Pruébalo cambiando el stock del Mouse.

Ejercicio 9 · Clasificar los productos por nivel de stock

Nivel: intermedio. Un diccionario cuyos valores son listas permite agrupar elementos por categoría; es una de las estructuras más usadas para organizar datos.

Enunciado. Escribe la función `clasificar_por_stock(inventario, minimo=5, maximo=10)` que devuelva un diccionario de listas con los nombres de los productos en tres grupos: `"bajo"` (stock menor que `minimo`), `"normal"` (stock entre `minimo` y `maximo`, ambos incluidos) y `"alto"` (stock mayor que `maximo`).

Conceptos que practicas: diccionario de listas, condicionales `if / elif / else`, parámetros por defecto y recorrido de un diccionario con `.items()`.

Solución

```
def clasificar_por_stock(inventario, minimo=5, maximo=10):
    niveles = {"bajo": [], "normal": [], "alto": []}
    for nombre, datos in inventario.items():
        stock = datos["stock"]
        if stock < minimo:
            niveles["bajo"].append(nombre)
        elif stock <= maximo:
            niveles["normal"].append(nombre)
        else:
            niveles["alto"].append(nombre)
    return niveles
```

Prueba

```
niveles = clasificar_por_stock(inventario)
print(niveles)

for nivel, nombres in niveles.items():
    print(f"{nivel}: {len(nombres)} producto(s)")

print(clasificar_por_stock(inventario, minimo=3, maximo=5))
```

Salida esperada

```
{'bajo': ['Teclado', 'Audifonos'], 'normal': ['Mouse', 'Monitor'], 'alto':
['Cable HDMI']}
bajo: 2 producto(s)
normal: 2 producto(s)
alto: 1 producto(s)
{'bajo': ['Audifonos'], 'normal': ['Teclado', 'Monitor'], 'alto': ['Mouse',
'Cable HDMI']}
```

Explicación línea por línea

Línea	Código	Qué hace y por qué
1	`def clasificar_por_stock(inventario, minimo=5, maximo=10):`	Define la función con dos umbrales opcionales. Con los valores por defecto, menos de 5 unidades es stock bajo y más de 10 es stock alto.
2	`niveles = {"bajo": [], "normal": [], "alto": []}`	Crea el diccionario con las tres claves y una lista vacía en cada una. Se crea antes del ciclo, para que las tres categorías existan aunque alguna quede vacía.
3	`for nombre, datos in inventario.items():`	Recorre cada producto; `datos` es el diccionario interno con precio y stock.
4	`stock = datos["stock"]`	Guarda el stock en una variable para escribir condiciones más cortas.
5	`if stock < minimo:`	Primera pregunta: ¿está por debajo del mínimo? Se usa `<` y no `<=`, porque el enunciado dice «menor que».
6	`niveles["bajo"].append(nombre)`	`niveles["bajo"]` es una lista, así que se le agrega el nombre con `.append()`. Es el patrón «diccionario de listas»: primero se busca la clave y luego se opera sobre la lista que contiene.

Línea	Código	Qué hace y por qué
7	`elif stock <= maximo:`	Solo se evalúa si la línea 5 fue falsa; por eso ya se sabe que `stock >= minimo`. Aquí se usa `<=` para que el máximo quede incluido en «normal».
8	`niveles["normal"].append(nombre)`	Agrega el nombre al grupo intermedio.
9	`else:`	Recoge todo lo demás: lo que no es bajo ni normal, es decir, `stock > maximo`.
10	`niveles["alto"].append(nombre)`	Agrega el nombre al grupo de stock alto.
11	`return niveles`	Devuelve el diccionario con los tres grupos, con la sangría del `def` para ejecutarse al final.

Qué ocurre con cada producto (con `minimo=5`, `maximo=10`)

Producto	Stock	Condición que se cumple	Grupo
Mouse	10	`10 < 5` no; `10 <= 10` sí	normal
Teclado	3	`3 < 5` sí	bajo
Monitor	5	`5 < 5` no; `5 <= 10` sí	normal
Audifonos	2	`2 < 5` sí	bajo
Cable HDMI	20	`20 < 5` no; `20 <= 10` no; cae en `else`	alto

Observa el Mouse y el Monitor: están justo en los límites y por eso se escriben `<` en la línea 5 y `<=` en la línea 7.

Error frecuente. Escribir tres `if` seguidos en lugar de `if / elif / else`. Con tres `if`, un mismo producto podría evaluarse tres veces y, según las condiciones, caer en más de un grupo. Con `elif` y `else`, solo entra en el primer grupo que cumpla.

Para pensar. ¿Cómo agregarías un cuarto grupo `"agotado"` para los productos con stock igual a 0? ¿Dónde irían la nueva clave y la nueva condición?

Ejercicio 10 · Cotizar un pedido (integrador)

Nivel: integrador. Este ejercicio junta todo lo anterior: una función calcula y devuelve varias estructuras de datos, y otra función se encarga solo de mostrarlas. Separar «calcular» de «mostrar» hace el código más fácil de probar y de reutilizar.

Enunciado. Un cliente envía un pedido como lista de tuplas `(producto, cantidad)`.

1. Escribe `cotizar_pedido(inventario, pedido, descuento=0)`. Para cada línea del pedido: si el producto no existe, se rechaza con el motivo `"No existe"`; si la cantidad supera el stock, se rechaza con `"Stock insuficiente"`; si no, se acepta. La función devuelve una tupla `(total, detalle, errores)`: `total` es la suma de las líneas aceptadas menos el `descuento` (un porcentaje); `detalle` es una lista de tuplas `(producto, cantidad, subtotal)`; `errores` es una lista de tuplas `(producto, cantidad, motivo)`. La función no modifica el inventario.
2. Escribe `imprimir_cotizacion(total, detalle, errores)` que muestre el resultado con formato.

Conceptos que practicas: validar con `in` y comparaciones, `if / elif / else`, listas de tuplas, devolver tres valores, parámetro por defecto, f-strings con formato y diferencia entre `return` y `print`.

Solución

```
def cotizar_pedido(inventario, pedido, descuento=0):
    subtotal = 0
    detalle = []
    errores = []
    for producto, cantidad in pedido:
        if producto not in inventario:
            errores.append((producto, cantidad, "No existe"))
        elif cantidad > inventario[producto]["stock"]:
            errores.append((producto, cantidad, "Stock insuficiente"))
        else:
            valor = cantidad * inventario[producto]["precio"]
            detalle.append((producto, cantidad, valor))
            subtotal += valor
    total = subtotal - subtotal * descuento / 100
    return total, detalle, errores
```

```
def imprimir_cotizacion(total, detalle, errores):
    print("=== COTIZACIÓN ===")
    for producto, cantidad, subtotal in detalle:
        print(f"  {producto} x{cantidad}: ${subtotal:,.0f}")
    print(f"Total a pagar: ${total:,.0f}")
    if errores:
        print("No se pudieron cotizar:")
        for producto, cantidad, motivo in errores:
            print(f"  - {producto} x{cantidad}: {motivo}")
```

Prueba

```
pedido = [("Mouse", 2), ("Teclado", 5), ("Parlante", 1), ("Cable HDMI", 4)]

total, detalle, errores = cotizar_pedido(inventario, pedido)
print(total)
print(detalle)
print(errores)

total, detalle, errores = cotizar_pedido(inventario, pedido, descuento=10)
imprimir_cotizacion(total, detalle, errores)
```

Salida esperada

```
110000.0
[('Mouse', 2, 50000), ('Cable HDMI', 4, 60000)]
[('Teclado', 5, 'Stock insuficiente'), ('Parlante', 1, 'No existe')]
=== COTIZACIÓN ===
  Mouse x2: $50,000
  Cable HDMI x4: $60,000
Total a pagar: $99,000
No se pudieron cotizar:
  - Teclado x5: Stock insuficiente
  - Parlante x1: No existe
```

Explicación línea por línea · `cotizar_pedido`

Línea	Código	Qué hace y por qué
1	`def cotizar_pedido(inventario, pedido, descuento=0):`	Define la función con tres parámetros. `descuento=0` es opcional: sin él, no se aplica rebaja.

Línea	Código	Qué hace y por qué
2	`subtotal = 0`	Acumulador numérico que sumará el valor de las líneas aceptadas.
3	`detalle = []`	Lista donde se guardarán las líneas aceptadas, como tuplas de tres datos.
4	`errores = []`	Lista donde se guardarán las líneas rechazadas, con su motivo. Las tres variables se inician antes del ciclo para que conserven lo acumulado.
5	`for producto, cantidad in pedido:`	Recorre el pedido y desempaqueta cada tupla de dos valores.
6	`if producto not in inventario:`	Primera validación: ¿el producto no es una clave del inventario? Debe ir primero, porque las comparaciones siguientes buscan `inventario[producto]` y fallarían con `KeyError` si no existe.
7	`errores.append((producto, cantidad, "No existe"))`	Registra el rechazo como una tupla de tres datos. Los paréntesis dobles son necesarios: los de afuera pertenecen a `.append()` y los de adentro crean la tupla.
8	`elif cantidad > inventario[producto]["stock"]:`	Segunda validación, que solo se evalúa si el producto existe. Entra en dos niveles del diccionario anidado para comparar la cantidad pedida con el stock.
9	`errores.append((producto, cantidad, "Stock insuficiente"))`	Registra el segundo tipo de rechazo.
10	`else:`	Si el producto existe y hay stock suficiente, la línea se acepta.
11	`valor = cantidad * inventario[producto]["precio"]`	Calcula el subtotal de la línea: cantidad por precio unitario.

Línea	Código	Qué hace y por qué
12	`detalle.append((producto, cantidad, valor))`	Guarda la línea aceptada como tupla de tres datos.
13	`subtotal += valor`	Suma el valor de la línea al acumulado general.
14	`total = subtotal - subtotal * descuento / 100`	Fuera del ciclo (sangría del `def`), porque se calcula una sola vez. Con `descuento=10` resta el 10 % del subtotal: 110000 − 11000 = 99000. Con `descuento=0` resta 0 y deja el subtotal.
15	`return total, detalle, errores`	Devuelve tres valores empaquetados en una tupla. La función no usa `print`: entrega datos para que otro código decida qué hacer con ellos.

Explicación línea por línea · `imprimir_cotizacion`

Línea	Código	Qué hace y por qué
1	`def imprimir_cotizacion(total, detalle, errores):`	Recibe los tres resultados de la función anterior, en el mismo orden en que ella los devuelve.
2	`print("=== COTIZACIÓN ===")`	Imprime el encabezado.
3	`for producto, cantidad, subtotal in detalle:`	Recorre las líneas aceptadas y desempaqueta cada tupla de tres datos.
4	`print(f" {producto} x{cantidad}: ${subtotal:,.0f}")`	Un f-string inserta variables entre llaves. `:,.0f` muestra el número con separador de miles y sin decimales. El `$` fuera de las llaves es un carácter normal.
5	`print(f"Total a pagar: ${total:,.0f}")`	Imprime el total ya con el descuento aplicado. Como `total` puede ser decimal, `.0f` lo redondea para mostrarlo.

Línea	Código	Qué hace y por qué
6	`if errores:`	Una lista vacía cuenta como falsa y una lista con elementos como verdadera. Así la sección de errores solo aparece si hubo rechazos.
7	`print("No se pudieron cotizar:")`	Imprime el título de la sección de rechazos.
8	`for producto, cantidad, motivo in errores:`	Recorre los rechazos y desempaqueta cada tupla de tres datos.
9	`print(f" - {producto} x{cantidad}: {motivo}")`	Imprime un rechazo por línea, con el producto, la cantidad y el motivo.

Seguimiento del pedido

Línea del pedido	¿Existe?	¿Alcanza el stock?	Resultado	Valor
`("Mouse", 2)`	Sí	2 ⩽ 10, sí	Aceptada	2 × 25000 = 50000
`("Teclado", 5)`	Sí	5 ⩽ 3, no	Rechazada: Stock insuficiente	no aplica
`("Parlante", 1)`	No	no se evalúa	Rechazada: No existe	no aplica
`("Cable HDMI", 4)`	Sí	4 ⩽ 20, sí	Aceptada	4 × 15000 = 60000

Subtotal: 50000 + 60000 = 110000. Con 10 % de descuento, el total es 99000. Observa que el `detalle` guarda los subtotales sin descuento; el descuento se aplica una sola vez, sobre el total.

Error frecuente. Invertir las validaciones y preguntar primero por el stock. Con `("Parlante", 1)` se ejecutaría `inventario["Parlante"]` y el programa se detendría con `KeyError`. El orden de las condiciones forma parte de la lógica.

Para pensar. Si el mismo producto aparece dos veces en el pedido, por ejemplo `("Mouse", 6)` y `("Mouse", 6)`, cada línea pasa la validación por separado (6 ⩽ 10), pero juntas suman 12 y superan el stock. ¿Cómo combinarías la función del ejercicio 6 con esta para detectar el problema?

Patrones para recordar y retos extra

Los 10 ejercicios repiten siete patrones; quien los domine puede resolver la mayoría de problemas de funciones con estructuras de datos de este nivel.

Los siete patrones

Patrón	Cómo se escribe	Ejercicios
Recorrer una lista de tuplas	`for nombre, precio, stock in productos:`	1, 2, 3, 6, 10
Recorrer un diccionario	`for clave, valor in diccionario.items():`	5, 7, 8, 9
Acumular en una lista	`lista = []` antes del ciclo y `lista.append(x)` dentro	1, 3, 5, 8, 9, 10
Acumular en un diccionario	`d[clave] = d.get(clave, 0) + cantidad`	6
Buscar sin que falle	`d.get(clave)` o `clave in d` antes de `d[clave]`	2, 4, 7, 10
Devolver varios valores	`return a, b, c` y recibirlos con `x, y, z = funcion()`	3, 10
No modificar el original	Crear diccionarios nuevos en vez de asignar sobre los existentes	7

Errores que más se repiten

- Poner el `return` dentro del ciclo `for`, con lo que la función termina en la primera vuelta.
- Crear la lista o el acumulador dentro del ciclo, con lo que se reinicia en cada vuelta.
- Buscar una clave con corchetes sin comprobar antes que existe, y recibir un `KeyError`.
- Usar `print` donde se necesita `return`, y recibir `None` al guardar el resultado.

- Copiar un diccionario anidado con `=` o con `.copy()` y modificar sin querer el original.

Retos extra

Son ejercicios sin solución en esta guía, para que el aprendiz los resuelva solo y los compare con el grupo.

1. `buscar_por_nombre(inventario, texto)`: devuelve la lista de nombres que contienen `texto`, sin importar mayúsculas ni minúsculas. Pista: `.lower()` y el operador `in` con cadenas.
2. `producto_mas_barato(inventario)`: devuelve una tupla `(nombre, precio)`, siguiendo el modelo de `producto_mas_caro` de la evaluación.
3. `actualizar_precios(inventario, cambios)`: `cambios` es un diccionario como `{"Mouse": 27000}`; devuelve un inventario nuevo con esos precios y sin tocar el original.
4. `resumen_ventas(inventario, ventas)`: devuelve un diccionario `{producto: {"unidades": n, "ingresos": total}}` solo con los productos que existen. Combina los ejercicios 6 y 10.
5. `promedio_por_nivel(inventario)`: usando `clasificar_por_stock`, devuelve el precio promedio de los productos de cada nivel de stock.